

Cargoservice — Sprint2

ISS 2026 — VALERIA CAPACCI & GINEVRA PENTOLI

[GOAL](#) [PROBLEM ANALYSIS](#) [TEST PLANS](#) [PROJECT](#) [DEPLOYMENT](#) [SPRINT3](#)

Goal

Progettare e realizzare `cargorobot`, il componente che orchestra il movimento del DDR. Il suo compito è portare il container dall'IOPort a `slot5` (dove avviene la marcatura), e da lì allo slot riservato, per poi tornare in HOME (requisito **R3** del committente). Il goal comprende inoltre la separazione dei Context per nodo fisico.

Problem analysis

Cargorobot come componente intermedio

Come stabilito già in Sprint0, `RobotSmart26` è il servizio, fornito dal committente, che `cargorobot` utilizza come servizio esterno per il movimento del DDR, scelto fra le alternative disponibili (`RobotObj26`, `RobotService26`, `RobotSmart26`) perché è quella che riduce maggiormente l'abstraction gap: calcola autonomamente il percorso verso un punto di cui sono note le coordinate. `cargorobot` resta comunque un'entità applicativa da progettare e realizzare, distinta da `RobotSmart26`, con la responsabilità di orchestrare la sequenza di operazioni. È quindi un componente intermedio fra `RobotSmart26` e `cargoservice`.

`RobotSmart26` conosce infatti solo primitive di movimento verso coordinate e non ha alcuna nozione dei concetti di dominio del sistema (IOPort, slot5, marker, stiva, sequenza di carico) né dello stato di avanzamento di un'operazione di carico. Se `cargoservice` invocasse `RobotSmart26` direttamente dovrebbe gestire sia la logica di business dell'orchestrazione delle varie richieste di carico sia la comunicazione col servizio esterno `RobotSmart26`, con due conseguenze indesiderate: la logica di orchestrazione del DDR resterebbe sparsa nell'attore `cargoservice` invece di essere incapsulata in un'unica entità responsabile e un cambiamento nel servizio robot si propagherebbe fino a `cargoservice` invece di restare confinato al solo componente

che lo usa. Introdurre `cargorobot` come entità intermedia mantiene invece la stessa separazione delle responsabilità, andando a rispettare il Single Responsibility Principle.

Comunicare con RobotSmart26

Analizzando il servizio fornito RobotSmart26 notiamo che esistono diversi tipi di comunicazione possibile. Dobbiamo quindi valutare e scegliere, fra i canali che il servizio espone, quello che riteniamo più adatto per far comunicare `cargoservice` con il servizio:

Alternativa	Considerazioni
ExternalQActor	si dichiara un <code>ExternalQActor</code> che punta al Context su cui gira RobotSmart26. Da quel momento <code>cargorobot</code> può inviare <code>request / dispatch</code> verso <code>robotsmart</code> esattamente come farebbe con un attore locale: la serializzazione dei messaggi e il loro instradamento in rete sono gestiti automaticamente dall'infrastruttura qak.
MQTT	RobotSmart26 può essere raggiunto anche pubblicando sulla topic <code>robotsmart26in</code> di un broker; gli eventi/reply generati vengono ripubblicati sulla topic gemella <code>robotsmart26in_out</code> . Introduce quindi un componente architetturale aggiuntivo e soprattutto, a differenza del canale ExternalQActor, i payload vanno costruiti e correlati esplicitamente.
CoAP	RobotSmart26 espone anche un client CoAP dedicato, pensato però per l'osservazione delle risorse (lo stato di <code>robotmnemo</code>) piuttosto che per l'invio di comandi; usarlo per pilotare il robot richiederebbe di reimplementare da zero sia il mapping dei comandi sia la logica di correlazione fra richiesta e risposta, senza alcun beneficio rispetto alle altre due alternative nel contesto di cargoservice.

Si decide di adottare l' `ExternalQActor` nativo qak come canale primario fra `cargorobot` e RobotSmart26 perché non introduce componenti aggiuntivi (a differenza di MQTT) e non richiede logica di correlazione scritta a mano (a differenza di CoAP). Essendo RobotSmart26 esso stesso realizzato in qak, questo canale non richiede inoltre alcun livello di conversione dei messaggi.

I comandi di RobotSmart26

RobotSmart26 espone il proprio comportamento a più livelli di astrazione, non uno solo, e va quindi scelto quale usare per orchestrare il percorso che devono seguire i

container. I comandi rilevanti sono:

- `moverobot(TARGETX,TARGETY,STEPTIME) → moverobotok(ARG) / moverobotfailed(PLANDONE,PLANTODO)` : muove il robot verso una coppia di coordinate assolute, calcolando autonomamente il percorso e orientandosi verso l'obiettivo prima di partire;
- `buildPlan(PX,PY,TX,TY) → buildPlanDone(PLAN)` seguito da `doplan(PLAN,STEPTIME) → doplandone(ARG) / doplanfailed(PLANTODO)` : separa il calcolo del piano dalla sua esecuzione, lasciando al chiamante la possibilità di ispezionare o modificare il piano fra i due passi;
- `step(TIME) → stepdone(V) / stepfailed(DURATION,CAUSE)` e `move(M)` (dispatch, $M = ||r||d|h$): comandi elementari di singolo passo/rotazione, pensati per scenari di esplorazione in cui la mappa non è nota a priori.

Analizzando le varie possibilità:

Alternativa	Considerazioni rispetto al caso d'uso
<code>moverobot</code>	massima riduzione dell'abstraction gap: <code>cargorobot</code> chiede solo "vai in (X,Y)", senza occuparsi di pianificazione. Adatta quando l'ambiente è noto a priori, come nel nostro caso.
<code>buildPlan</code> + <code>doplan</code>	utile se occorre ispezionare o adattare il piano prima di eseguirlo (ad esempio per evitare una zona); introduce però in <code>cargorobot</code> una responsabilità di orchestrazione del piano che, per le nostre tratte fisse, non ha controparte in alcun requisito.
<code>step</code> / <code>move</code>	livello più basso, pensato per l'esplorazione di un ambiente non ancora mappato; nel nostro caso l'ambiente è già interamente noto (le coordinate degli slot sono un dato di progetto), quindi usarlo significherebbe reimplementare in <code>cargorobot</code> una logica di path-planning che RobotSmart26 offre già.

Si sceglie `moverobot` come comando primario di movimento: le tratte che `cargorobot` deve percorrere sono verso coordinate fisse e note in anticipo, per cui non c'è alcun bisogno di intervenire sul piano calcolato.

Il marker

Il requisito descrive il marker come un dispositivo a sé, ma vale la pena chiedersi se avrebbe senso realizzarlo come una singola entità separata oppure se "inglobarlo" assieme a `cargoservice`. Analizzandone il comportamento notiamo infatti che esso si

occupa solo di gestire per ogni container che gli viene affidato un timeout (con cui notifica il DDR) e l'assegnazione dello slot. Avendo quindi minime responsabilità, ha pertanto senso gestirlo internamente all'orchestratore in modo "astratto", trattandolo come dettaglio implementativo.

Sempre a proposito del marker, è necessario anche stabilire come il DDR viene informato della terminazione della marcatura e, di conseguenza, del suo spostamento verso lo slot 5. Per la comunicazione sono possibili due strade:

Alternativa	Considerazioni
Polling	<code>cargorobot</code> interroga periodicamente <code>cargoservice</code> prima di prelevare. Attesa attiva, nessun beneficio rispetto a un modello reattivo.
Notifica su richiesta bloccante	<code>cargorobot</code> , depositato il container, invia una richiesta e resta in attesa della risposta; <code>cargoservice</code> risponde solo dopo lo scadere del timeout.

Si decide di optare per la seconda opzione: a `cargorobot` basta solo ricevere la reply a tempo debito.

Comportamento di `cargorobot` al guasto di `RobotSmart26`

Come stabilito precedentemente con il committente, in presenza di guasti il robot si ferma immediatamente e resta fermo finché il sistema non decide il da farsi.

`cargorobot` si limita quindi a riconoscere il fallimento e a fermarsi, mentre la decisione su come reagire resta a `cargoservice`.

Separazione dei Context per nodo fisico

Precedentemente si era già individuata la necessità di almeno 3-4 nodi fisici distinti (`cargorobot`, `cargoservice`, `sonar`, e `IOPort` se trattato a sé); Sprint1 aveva mantenuto `cargoservice` e `ioport` in un unico Context per pura semplicità, rimandando esplicitamente la separazione reale a questo sprint.

Si introduce quindi un Context per ogni nodo fisico realmente distinto, coerentemente col fatto che ogni nodo userà tecnologie diverse e con quanto richiesto dal goal di questo sprint. I nodi introdotti in questo sprint sono:

- `ctxcargoservice` (`localhost:8050`, invariata da Sprint0/1): ospita l'attore `cargoservice`, `Hold` (POJO) e il marker.

- **ctxioport** (nuova, es. `localhost:8060`): ospita l'attore **ioport** e l'adapter **IOPortWsAdapter** verso la WebGUI, separato ora da **cargoservice**.
- **ctxcargorobot** (nuova, es. `localhost:8070`): ospita l'attore **cargorobot**.
- **RobotSmart26**: già esterno per definizione (fornito dal committente), referenziato come **ExternalQActor** nel Context di **cargorobot**, su `localhost:8020`.

Di conseguenza **cargoservice**, per parlare con **cargorobot** o con **ioport**, lo dichiara **ExternalQActor** nel proprio Context.

Test plans

Test che guideranno l'implementazione di **cargorobot**.

Test	Obiettivo
Percorso completo del container	Verificare che cargorobot , ricevuto l'incarico di caricare un container, invii correttamente le richieste moverobot , rispettando l'ordine richiesto dal requisito R3 (IOPort → slot5 → slot riservato → HOME).
Successo del movimento	Simulare la risposta moverobotok da parte di RobotSmart26 (o di uno stub che la replica) e verificare che cargorobot proceda correttamente al passo successivo della sequenza.
Fallimento del movimento	Simulare la risposta moverobotfailed e verificare che cargorobot si fermi immediatamente e resti in attesa, senza inviare ulteriori comandi finché il sistema non decide come procedere.
Notifica di fine marcatura	Verificare che, depositato il container in slot5 , cargorobot resti bloccato in attesa della reply di cargoservice e riprenda il movimento solo dopo la ricezione della notifica di scadenza del timeout.
Comunicazione fra Context	Verificare che i messaggi scambiati fra ctxcargoservice , ctxioport e ctxcargorobot tramite gli ExternalQActor arrivino correttamente a destinazione, eseguendo i tre Context come processi realmente separati.

Project

Prima di scrivere codice, si attende un feedback del committente sui punti di analisi.

Deployment

Prima di scrivere codice, si attende un feedback del committente sui punti di analisi.

Definizione dello Sprint3

Lo Sprint3 sarà l'ultimo sprint del progetto.

REQUISITI

- Progettare e realizzare l'attore `sonar` : scrivere il firmware del microcontrollore Raspberry che misura la distanza `D` davanti all'IOPort e genera gli eventi `sonarcontainer` (container rilevato) e `sonarfault` (guasto, $D > D_{FREE}$ per almeno 3 s), coerentemente con quanto già modellato in Sprint0.
- Gestire la reazione del sistema al guasto sonar durante il movimento del DDR: fermata immediata (*halt*) di `cargorobot` e ripresa dell'operazione dal punto di interruzione al ripristino ($D \leq D_{FREE}$), come deciso con il committente.
- Completare la gestione del `LED` , sostituendo il `println` provvisorio con l'effettiva interazione con il microcontrollore (comando `blinkLed`).
- Dare a `sonar` un Context proprio, finora rimasto solo un placeholder concettuale dentro `ctxcargoservice` , completando così la separazione dei nodi fisici avviata nello Sprint2.



Valeria Capacci

valeria.capacci@studio.unibo.it



Ginevra Pentoli

ginevra.pentoli@studio.unibo.it

GIT: github.com/valecapa/TemaFinaleISS2026